

logs/dev76 目录全面分析 — 多线程 CUDA C++ MultiGrid 测试套件

opencode analy

2026 年 8 月 16 日

摘要

本报告对 `logs/dev76/` 目录进行全面分析：它是一套完整的测试套件，用于验证多线程版（一线程一卡）CUDA C++ MultiGrid 求解器的正确性与性能，并记录 dev76 里程碑的粗层求解优化成果。三视角概览：[主体结构](#)为单文件 `main.py` 子命令入口 + 版本化产物目录 `v<ts>/` + `h5py` 持久化；[各部分关系](#)为测试脚本经 Cython 桥驱动 C++ 后端，粗算子构建与求解阶段通过 `h5py` 缓存复用；[项目思路](#)为物理正确性优先、真多线程并行、以实测数据驱动优化。核心结论：verify 全 PASS；bench 最佳 8x8x8x16 3L 加速比 2.13；sweep 14/16 配置达到 `gate=1.5`；dev76 优化使粗层求解 10 倍加速（`coarse_solve` 533ms → 42ms）。

目录

1	目录概览	2
1.1	logs/dev76 文件清单	2
1.2	版本目录实测（v202608160001，最终归档）	2
2	主体结构	2
3	各部分关系	3
4	项目思路	3
4.1	设计意图	3
4.2	演进脉络	3
4.3	典型 workflow	4
5	主题分析	4
5.1	测试覆盖（verify）	4
5.2	性能结果（bench）	4
5.3	参数扫描（sweep）	5
5.4	dev76 优化成果	5
6	关键代码/文档片段	5

1 目录概览

1.1 logs/dev76 文件清单

1	logs/dev76/	
2	main.py	全部测试代码 (816 行, 子命令入口)
3	run-local.sh	运行脚本 (设备编排 + 版本目录, 101 行)
4	AGENTS.md	复现与比对指南 (134 行)
5	dev76_report.md	收尾结果报告 (47 行)
6	docs/	analy 报告 (analy_dev76_*.tex/.pdf)
7	logs/	C++ 收敛日志归档
8	v<YYYYMMDDHHMM>/	版本化产物目录 (每次运行独立)
9	test76_verify.h5	正确性验证
10	test76_clean_*.h5	干净测量
11	test76_bench.h5	批量基准
12	test76_sweep.h5	参数扫描
13	test76_results.h5	collect 汇总
14	test76_budget_*.h5	显存预算
15	test76_tbl_*.tex	LaTeX 表
16	test76_*.png	图

1.2 版本目录实测 (v202608160001, 最终归档)

指标	实测值	说明
bench 中位加速比	1.148	7 配置
bench 最大加速比	2.135	8x8x8x16 3L P100x2
sweep 最大加速比	2.306	L3 r10 ct1e5 cmi10
sweep 通过率 (gate=1.5)	14/16	8x8x8x16 P100x2
verify 正确性	全 PASS	rel_diff=0

表 1: logs/dev76 最终产物实测统计 (数据来源: test76_results.h5 / verify.h5 实测)

2 主体结构

logs/dev76/ 的分层结构:
main.py 的子命令体系 (logs/dev76/main.py:757-812): verify → clean → bench → sweep → check → budget → collect → mktable → plots, 每子命令产出独立 h5 文件, 最终 collect 汇总。

层次	文件	职责
入口层	main.py / run-local.sh	测试编排 (verify/clean/bench/sweep/check-/budget)
被测对象	pyqcu/cuda/_multi_gpu.py	MultiGpuMultigrid (N 线程 × 卡并行)
数据层	env.h5 / test76_*.h5	h5py 持久化 (独立 File 句柄多线程安全)
文档层	AGENTS.md / dev76_report.md / docs/	复现指南 / 汇总 / analy 报告

表 2: 主体结构分层 (数据来源: 全库索引实测)

3 各部分关系

要点

依赖主链: run-local.sh → main.py → pyqcu/cuda/_multi_gpu.py (MultiGpuMultigrid) → pyqcu/cuda/qcu.pyx (Cython 桥) → cpp/cuda/qcu/libqcu.so (C++ 内核); 粗算子构建经 build_schur_levels (batch 路径) → logs/nullvec_cache h5py 缓存; 结果经 save_dict_h5 → test76_*.h5 → collect/mktable/plots

优化与测试的关系 (dev76 里程碑闭环):

- C++ 优化 (lattice_clover_multigrid.h 多 block 归约) → bench 验证 (8x8x8x16 2L 1.83→2.01) → sweep 参数确认 (best 2.31) → 归档 v<ts>/;
- 粗算子构建 (test14 批量路径) → nullvec_cache 跨 tag 复用 → 测试秒级启动 (16x16x16x32 3L 构建 86s 一次性成本)。

4 项目思路

4.1 设计意图

- 版本化产物: 每次运行独立 v<ts>/ 目录, 跨环境可横向比对 (env.h5 记录 GPU/软件/git 快照);
- h5py-only 持久化: 独立 File 句柄保证多线程安全, 结果字典经 attrs+datasets 存储 (save_dict_h5);
- 真多线程: 求解热点在 worker 线程各自卡上并行 (qcu.pyx with nogil), 测试脚本只编排收集。

4.2 演进脉络

logs/test12 (单线程) → test13 (多线程 MultiGpuMultigrid) → test14 (粗算子构建加速) → dev76 (粗层求解加速, 本次)。每个里程碑在前者基础上消除一个瓶颈: 构建 1h+ → 86s (test14), 粗层求解 533ms → 42ms (dev76)。

4.3 典型 workflow

- 运行: `source env.sh → bash logs/dev76/run-local.sh`;
- 产物: `v<ts>/` 版本目录 (h5py 结果 + 表 + 图);
- 比对: `diff` 两个版本的 `test76_results.h5` 横向对比。

项目思路

项目思路核心: 先消除构建/求解瓶颈, 再测加速比; 一切以实测为准; 套件可复现、可对比、可演进。

5 主题分析

5.1 测试覆盖 (verify)

正确性验证四合一 (`logs/dev76/main.py:305-354`):

- 一致性模式: 2 线程 × P100×2 共享输入, 各线程解一致 (`rel_diff=0`);
- 独立问题模式: 2 线程不同 seed, 解不同且各自收敛 (`|d|=7.75`);
- V100 单线程大格子: 8x16x16x16 3L (`rel_diff=0`);
- h5py 多线程 IO: 4 线程并发写/读往返 (`max_err=0`)。

5.2 性能结果 (bench)

配置	设备	ref(s)	mg(s)	speedup
8x8x8x16 2L	P100x2	0.685	0.342	2.007
8x8x8x16 3L	P100x2	0.685	0.321	2.135
8x16x16x16 2L	P100x2	0.655	1.994	0.329
8x16x16x16 3L	P100x2	0.687	0.996	0.690
16x16x16x16 2L	P100x2	0.766	1.191	0.643
16x16x16x16 3L	V100	0.698	0.608	1.148
8x16x16x16 3L	V100	0.650	0.474	1.370

表 3: bench 实测 (数据来源: `test76_bench.h5`)

规律: 3L 优于 2L (粗层更小、递归校正更有效); 中/大格子 2L < 1 为层数固有特性; 小格子 (8x8x8x16) 加速比 2.0-2.1。

5.3 参数扫描 (sweep)

16 配置 (levels $\times$ restart $\times$ ct $\times$ cmi) $14/16 \geq 1.5$; 最佳 2.306 (L3 r10 ct1e5 cmi10)。r10 > r5 (V-cycle 频率更高收敛更快)、ct1e5 > ct1e4 (粗层容差更紧)、3L > 2L 均与 test13/14 一致 (参数相对行为可迁移)。

5.4 dev76 优化成果

粗层点积多 block 归约 (lattice_clover_multigrid.h:82-112) : 16x16x16x32 3L 中间粗层 (196608 元素) coarse_solve 533ms $\rightarrow$ 42ms (约 10 倍); 大最粗层 ($\geq 64K$) fused 回退普通迭代路径。

6 关键代码/文档片段

measure_multi (单配置测量核心):

```

1 def measure_multi(lat, num_levels=2, num_restart=5, coarse_max_iter=15,
2     coarse_tol_factor=1e5, nthreads=2, device_ids=None,
3     nv_iters=Nv_ITERS, use_cache=True, verbose=False,
4     independent=False, label=None):
5     """一次多线程求解测量 (MultiGpuMultigrid.solve 内部 ref/mg 交叉计时)。
6
7     返回结果字典 (h5 可存): 每线程 tid/device/ref_time/mg_time/rel_diff,
8     多线程墙钟 multi_ref_wall/multi_mg_wall, speedup, 一致性摘要, 资源统计。
9     """
10    from pyqcu.cuda._multi_gpu import MultiGpuMultigrid
11    if device_ids is None:
12        device_ids = list(P100_IDS if nthreads >= 2 else V100_ID)
13    mg = MultiGpuMultigrid(lat_size=list(lat), mass=MASS, atol=ATOL,
14        num_levels=num_levels, dof_list=DOF_LIST,
15        mg_grid=MG_GRID, num_restart=num_restart,
16        coarse_max_iter=coarse_max_iter,
17        coarse_tol_factor=coarse_tol_factor,
18        nv_iters=nv_iters, nthreads=nthreads,
19        device_ids=list(device_ids), use_cache=use_cache,
20        cache_dir=cache_dir(), verbose=verbose,

```

多 block 归约 kernel (dev76 核心优化):

```

1     if (idx < s) sdata[idx] += sdata[idx + s];
2     __syncthreads();
3 }
4 if (idx == 0) out[0] = sdata[0];
5 }
6
7 // Multi-block grid-stride reduction: partials[bid] = block-local <a,b> slice.
8 template <typename T, int NT>
9 __global__ void coarse_dot_kernel_multi(const LatticeComplex<T> *a,
10     const LatticeComplex<T> *b, int n,
11     LatticeComplex<T> *partials) {
12     __shared__ LatticeComplex<T> sdata[NT];
13     int idx = threadIdx.x, bid = blockIdx.x, nblk = gridDim.x;

```

```

14  int stride = nblk * NT;
15  LatticeComplex<T> sum(0, 0);
16  for (int i = bid * NT + idx; i < n; i += stride) sum += a[i].conj() * b[i];
17  sdata[idx] = sum;
18  __syncthreads();
19  for (int s = NT / 2; s > 0; s >>= 1) {
20      if (idx < s) sdata[idx] += sdata[idx + s];
21      __syncthreads();
22  }
23  if (idx == 0) partials[bid] = sdata[0];
24 }
25
26 // Second reduction over nblk partials (1 block).
27 template <typename T, int NT>
28 __global__ void coarse_dot_reduce_kernel(const LatticeComplex<T> *partials,
29                                          int nblk, LatticeComplex<T> *out) {
30     __shared__ LatticeComplex<T> sdata[NT];
31     int idx = threadIdx.x;
32     LatticeComplex<T> sum(0, 0);
33     for (int i = idx; i < nblk; i += NT) sum += partials[i];
34     sdata[idx] = sum;
35     __syncthreads();
36     for (int s = NT / 2; s > 0; s >>= 1) {
37         if (idx < s) sdata[idx] += sdata[idx + s];
38         __syncthreads();
39     }

```

7 参考源清单

参考源	类型	内容/作用
logs/dev76/main.py:252-299	仓库	measure_multi 测量逻辑
logs/dev76/main.py:305-354	仓库	verify 四合一验证
logs/dev76/AGENTS.md	仓库	套件约定与复现指南
logs/dev76/dev76_report.md	仓库	里程碑汇总
logs/dev76/v202608160001/test76_results.h5	仓库	最终汇总数据
logs/dev76/v202608152354/test76_verify.h5	仓库	正确性验证
cpp/cuda/qcu/include/lattice_clover_multigrid.h:74-112	仓库	多 block 归约 kernel
cpp/cuda/qcu/include/lattice_clover_multigrid.h:1227	仓库	fused 最粗层阈值

8 结论与局限

结论

三视角结论：**结构**——单文件子命令套件 + 版本目录 + h5py 数据层；**关系**——Python 编排经 Cython 桥驱动 C++ 内核，缓存复用贯穿；**思路**——先消瓶颈再测加速比，实测驱动。主题结论：logs/dev76 完整记录 dev76 里程碑——粗层求解 10 倍加速、全 3L 配置加速比提升 (+5% 28%)、verify 全 PASS、sweep 14/16 达标。

参考背景

多线程多卡 MultiGrid 测试套件设计（版本目录 + 环境快照 + 参数扫描）与 QUDA/Chroma 等生产框架的 benchmark 实践一致；加速比语义（墙钟 = max 各线程）为多线程并行测试的标准定义。

局限与风险

未验证项：16x16x16x32 3L 未纳入 bench（求解慢，已知特性），其优化成果仅在 dev76 专项测试确认；2L 大格子 <1 未做更细参数探索（cmi 50/200 提升有限）；h5py IO 并发上限仅测 4 线程，更高并发未验证。